

Guía Maestra de Patrones de Código para Programación (DAM/DAW)

Autor: Manus AI

Introducción

Esta guía está diseñada para proporcionarte patrones de código eficientes y fáciles de recordar, que te ayudarán a dominar los conceptos clave de las unidades didácticas UT1 a UT6, cubriendo los Resultados de Aprendizaje RA1, RA2 y RA3. Nos centraremos en Python, un lenguaje versátil y ampliamente utilizado, ideal para comprender los fundamentos de la programación y aplicar estructuras de control y manejo de datos.

Cada patrón incluirá una breve explicación, un ejemplo de código claro y conciso, y consejos para su aplicación en diferentes escenarios de examen. El objetivo es que puedas reconocer rápidamente el problema y aplicar la solución más adecuada de forma eficiente.

UT1 / UT2: Introducción a la Programación y Elementos de un Programa Informático (RA1, RA2)

Estas unidades sientan las bases de la programación. Es crucial entender la estructura básica de un programa, cómo declarar y usar variables, los tipos de datos, operadores y cómo interactuar con el usuario.

Patrón 1.1: Estructura Básica de un Programa y Comentarios

Todo programa Python sigue una estructura lógica. Es fundamental incluir comentarios para hacer el código legible y documentado.

```

# Importación de módulos (si es necesario)
import math # Ejemplo: Importar el módulo math

# Definición de constantes (se suelen escribir en MAYÚSCULAS)
PI = 3.14159
GRAVEDAD = 9.81

# Definición de funciones (bloques de código reutilizables)
def saludar(nombre):
    """Esta función saluda a una persona por su nombre."""
    print(f"¡Hola, {nombre}!")

# Bloque principal del programa (punto de entrada)
if __name__ == "__main__":
    # Comentario de una sola línea
    print("Este es el inicio de mi programa.")
    saludar("Mundo")

    # Uso de constantes
    radio = 5
    area_circulo = PI * (radio ** 2)
    print(f"El área de un círculo con radio {radio} es: {area_circulo:.2f}")

```

Explicación:

- `import` : Permite usar funcionalidades de otros módulos.
- Constantes: Valores fijos, por convención en mayúsculas.
- Funciones: Encapsulan lógica, mejorando la modularidad. Las cadenas de texto entre triples comillas (`"""Docstring"""`) son docstrings que explican la función.
- `if __name__ == "__main__":` : Asegura que el código dentro de este bloque solo se ejecute cuando el script se corre directamente.
- Comentarios (`#`): Explican partes del código.

Patrón 1.2: Declaración de Variables, Tipos de Datos y Operadores

Las variables almacenan información. Python es de tipado dinámico, pero es vital entender los tipos de datos para operar correctamente con ellos.

```

# Variables numéricas
entero = 10
flotante = 3.14

# Variables de texto (string)
cadena_simple = 'Hola'
cadena_doble = "Mundo"
cadena_multilinea = """Esta es una
cadena de varias
líneas."""

# Variables booleanas
es_verdadero = True
es_falso = False

# Operadores aritméticos
suma = entero + flotante # 10 + 3.14 = 13.14
resta = entero - 5        # 10 - 5 = 5
multiplicacion = flotante * 2 # 3.14 * 2 = 6.28
division = entero / 3      # 10 / 3 = 3.333...
division_entera = entero // 3 # 10 // 3 = 3
modulo = entero % 3        # 10 % 3 = 1 (resto de la división)
potencia = 2 ** 3          # 2 elevado a 3 = 8

print(f"Suma: {suma}, Resta: {resta}, Multiplicación: {multiplicacion}")
print(f"División: {division:.2f}, División Entera: {division_entera},
Módulo: {modulo}, Potencia: {potencia}")

# Operadores de comparación (devuelven booleanos)
es_igual = (entero == 10) # True
es_diferente = (flotante != 3.0) # True
es_mayor = (entero > flotante) # True
es_menor_o_igual = (5 <= 5) # True

print(f"Es igual: {es_igual}, Es diferente: {es_diferente}, Es mayor:
{es_mayor}, Es menor o igual: {es_menor_o_igual}")

# Operadores lógicos
logico_and = es_verdadero and es_falso # False
logico_or = es_verdadero or es_falso   # True
logico_not = not es_verdadero           # False

print(f"AND: {logico_and}, OR: {logico_or}, NOT: {logico_not}")

# Conversión de tipos (Casting)

```

```
numero_str = "123"
numero_int = int(numero_str) # Convierte string a entero

numero_float_str = "45.67"
numero_float = float(numero_float_str) # Convierte string a flotante

print(f"String a int: {numero_int}, String a float: {numero_float}")
```

Explicación:

- **Numéricos:** `int` para enteros, `float` para decimales.
- **Cadenas:** `str` para texto, se pueden usar comillas simples, dobles o triples.
- **Booleanos:** `bool` para `True` o `False`.
- **Operadores:** Conocer su función y precedencia es clave.
- **Casting:** Convertir un tipo de dato a otro (`int()`, `float()`, `str()`, `bool()`).

Patrón 1.3: Entrada y Salida de Datos (Interacción con el Usuario)

La interacción básica con el usuario se realiza mediante `input()` para la entrada y `print()` para la salida.

```

# Entrada de datos
nombre_usuario = input("Por favor, introduce tu nombre: ")
edad_str = input("Introduce tu edad: ")

# Es crucial convertir la entrada de input() si esperamos un número
try:
    edad = int(edad_str)
    print(f"Hola, {nombre_usuario}. Tienes {edad} años.")
except ValueError:
    print(f"Hola, {nombre_usuario}. La edad introducida '{edad_str}' no es un número válido.")

# Salida de datos con formato (f-strings)
producto = "Libro"
precio = 19.99
cantidad = 2
total = precio * cantidad

print(f"Has comprado {cantidad} unidades de {producto} por un total de {total:.2f} euros.")
# El :.2f formatea el número flotante a dos decimales.

```

Explicación:

- `input()` : Siempre devuelve una cadena de texto. Si necesitas un número, debes convertirlo explícitamente (e.g., `int()`, `float()`).
- `print()` : Muestra información en la consola. Las f-strings (`f"..."`) son la forma más moderna y legible de formatear cadenas.
- `try-except` : Es una buena práctica usarlo al convertir entradas de usuario para manejar errores si el usuario no introduce el tipo de dato esperado.

Patrón 1.4: Funciones Sencillas

Las funciones permiten organizar el código en bloques lógicos y reutilizables.

```
def sumar(a, b):
    """Esta función toma dos números y devuelve su suma."""
    resultado = a + b
    return resultado

def es_par(numero):
    """Verifica si un número es par."""
    return numero % 2 == 0

if __name__ == "__main__":
    num1 = 15
    num2 = 7

    suma_total = sumar(num1, num2)
    print(f"La suma de {num1} y {num2} es: {suma_total}")

    if es_par(num1):
        print(f"{num1} es un número par.")
    else:
        print(f"{num1} es un número impar.")

    if es_par(num2 + 1):
        print(f"{num2 + 1} es un número par.")
    else:
        print(f"{num2 + 1} es un número impar.")
```

Explicación:

- `def nombre_funcion(parametros):` : Define una función.
- `return` : Devuelve un valor de la función. Si no se especifica, la función devuelve `None`.
- Las funciones promueven la reutilización y la legibilidad del código.

Patrón 1.5: Condicionales Básicos (`if`, `elif`, `else`)

Permiten que el programa tome decisiones basadas en condiciones.

```
def verificar_edad(edad):  
    if edad < 18:  
        print("Eres menor de edad.")  
    elif edad >= 18 and edad < 65:  
        print("Eres adulto.")  
    else:  
        print("Eres un adulto mayor.")  
  
if __name__ == "__main__":  
    mi_edad = 25  
    verificar_edad(mi_edad)  
  
    otra_edad = 70  
    verificar_edad(otra_edad)  
  
    edad_nino = 10  
    verificar_edad(edad_nino)
```

Explicación:

- `if condicion:` : Ejecuta un bloque de código si la condición es verdadera.
- `elif condicion:` : (else if) Ejecuta un bloque si la condición anterior fue falsa y esta es verdadera.
- `else:` : Ejecuta un bloque si ninguna de las condiciones anteriores fue verdadera.

Patrón 1.6: Bucles Iterativos (for)

El bucle `for` se usa para iterar sobre una secuencia (listas, cadenas, rangos, etc.).

```

# Iterar sobre una lista
frutas = ["manzana", "banana", "cereza"]
print("\nFrutas en la lista:")
for fruta in frutas:
    print(f"- {fruta}")

# Iterar sobre un rango de números
print("\nNúmeros del 0 al 4:")
for i in range(5): # range(5) genera 0, 1, 2, 3, 4
    print(i)

# Iterar con índice y valor (enumerate)
print("\nFrutas con su índice:")
for indice, fruta in enumerate(frutas):
    print(f"El índice {indice} corresponde a {fruta}")

# Iterar sobre una cadena
nombre = "Python"
print("\nCaracteres del nombre:")
for caracter in nombre:
    print(caracter)

```

Explicación:

- `for elemento in secuencia:` : Recorre cada elemento en la secuencia .
- `range(n)` : Genera una secuencia de números desde 0 hasta `n-1` .
- `enumerate()` : Útil para obtener tanto el índice como el valor al iterar.

Patrón 1.7: Bucles Condicionales (`while`)

El bucle `while` se ejecuta mientras una condición sea verdadera.


```
# Bucle while simple
contador = 0
print("\nContando hasta 3 con while:")
while contador < 3:
    print(contador)
    contador += 1 # Es crucial actualizar la condición para evitar bucles
infinitos

# Bucle while para pedir entrada hasta que sea válida
contrasena_correcta = "secreto"
contrasena_introducida = ""

while contrasena_introducida != contrasena_correcta:
    contrasena_introducida = input("Introduce la contraseña: ")
    if contrasena_introducida != contrasena_correcta:
        print("Contraseña incorrecta. Inténtalo de nuevo.")

print("¡Contraseña correcta! Acceso concedido.")
```

Explicación:

- `while condicion:` : El bloque de código se repite mientras la `condicion` sea `True`.
- Asegúrate de que la condición del bucle `while` eventualmente se vuelva `False` para evitar bucles infinitos.

Continuaremos con los patrones para UT3/UT4 en la siguiente sección. Este es un buen punto de partida para RA1 y RA2.

UT3: Estructuras de Control (RA3 - Parte I)

Las estructuras de control son el corazón de la lógica de un programa, permitiendo ejecutar código de forma condicional o repetitiva. Dominarlas es fundamental para el RA3.

Patrón 3.1: Condicionales Anidados y Múltiples Condiciones

Permiten tomar decisiones más complejas combinando `if`, `elif`, `else` y operadores lógicos.

```

def clasificar_numero(numero):
    if numero > 0:
        if numero % 2 == 0:
            print(f"{numero} es positivo y par.")
        else:
            print(f"{numero} es positivo e impar.")
    elif numero < 0:
        print(f"{numero} es negativo.")
    else:
        print(f"{numero} es cero.")

def verificar_acceso(usuario, contrasena, rol):
    if usuario == "admin" and contrasena == "pass123":
        if rol == "administrador":
            print("Acceso concedido como administrador.")
        else:
            print("Acceso concedido, pero no eres administrador.")
    elif usuario == "invitado":
        print("Acceso concedido como invitado.")
    else:
        print("Credenciales incorrectas.")

if __name__ == "__main__":
    clasificar_numero(7)
    clasificar_numero(-4)
    clasificar_numero(0)
    clasificar_numero(10)

    print("\n")
    verificar_acceso("admin", "pass123", "administrador")
    verificar_acceso("admin", "pass123", "usuario")
    verificar_acceso("invitado", "", "")
    verificar_acceso("otro", "123", "")

```

Explicación:

- **Anidamiento:** Un `if` dentro de otro `if` permite evaluar condiciones secuenciales.
- **Múltiples condiciones:** Se usan `and`, `or`, `not` para combinar condiciones en una sola línea, haciendo el código más conciso.

Patrón 3.2: Control de Bucles (break y continue)

`break` y `continue` permiten modificar el flujo normal de los bucles `for` y `while`.

```
# Uso de break: salir del bucle completamente
print("\n--- Uso de break ---")
for i in range(10):
    if i == 5:
        print("Se encontró el 5, saliendo del bucle.")
        break
    print(i)

# Uso de continue: saltar la iteración actual
print("\n--- Uso de continue ---")
for i in range(10):
    if i % 2 != 0: # Si es impar
        print(f"Saltando número impar: {i}")
        continue
    print(f"Número par: {i}")

# Bucle while con break
print("\n--- Bucle while con break ---")
while True:
    respuesta = input("Escribe 'salir' para terminar: ").lower()
    if respuesta == "salir":
        print("Saliendo del programa.")
        break
    print(f"Has escrito: {respuesta}")
```

Explicación:

- `break`: Termina el bucle más interno y la ejecución continúa en la instrucción inmediatamente posterior al bucle.
- `continue`: Salta el resto del código de la iteración actual y pasa a la siguiente iteración del bucle.

Patrón 3.3: Bucles Anidados

Un bucle dentro de otro bucle es útil para trabajar con estructuras bidimensionales o combinaciones.

```
# Imprimir una tabla de multiplicar simple
print("\n--- Tabla de Multiplicar (Bucles Anidados) ---")
for i in range(1, 4): # Filas
    for j in range(1, 4): # Columnas
        print(f"{i} x {j} = {i*j}", end="\t") # end="\t" para tabular
    print() # Salto de línea después de cada fila

# Recorrer una matriz (lista de listas)
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print("\n--- Recorriendo una Matriz ---")
for fila in matriz:
    for elemento in fila:
        print(elemento, end=" ")
    print()
```

Explicación:

- El bucle interno se ejecuta completamente por cada iteración del bucle externo.
- Son comunes para procesar datos en tablas, matrices o generar combinaciones.

UT4: Estructuras de Datos (RA3 - Parte II)

Las estructuras de datos permiten organizar y almacenar información de manera eficiente. Python ofrece varias estructuras integradas muy potentes.

Patrón 4.1: Listas (Colecciones Ordenadas y Modificables)

Las listas son colecciones de elementos que pueden ser de diferentes tipos, ordenadas y mutables.

```

# Creación de listas
lista_vacia = []
frutas = ["manzana", "banana", "cereza", "dátil"]
numeros = [1, 5, 2, 8, 3]
mixta = ["texto", 10, True, 3.14]

print(f"\n--- Listas ---")
print(f"Lista de frutas: {frutas}")

# Acceso a elementos (por índice, empieza en 0)
primera_fruta = frutas[0] # "manzana"
ultima_fruta = frutas[-1] # "dátil"

print(f"Primera fruta: {primera_fruta}, Última fruta: {ultima_fruta}")

# Modificación de elementos
frutas[1] = "kiwi" # Cambia "banana" por "kiwi"
print(f"Lista modificada: {frutas}")

# Métodos comunes de listas
frutas.append("mango") # Añadir al final
print(f"Después de append: {frutas}")

frutas.insert(1, "pera") # Insertar en posición específica
print(f"Después de insert: {frutas}")

frutas.remove("cereza") # Eliminar por valor (primera ocurrencia)
print(f"Después de remove: {frutas}")

fruta_eliminada = frutas.pop() # Eliminar y devolver el último elemento
print(f"Después de pop (último): {frutas}, Eliminado: {fruta_eliminada}")

frutas.sort() # Ordenar la lista (modifica la lista original)
print(f"Lista ordenada: {frutas}")

# Slicing (obtener sublistas)
sub_lista = frutas[1:3] # Elementos desde el índice 1 hasta antes del 3
print(f"Sublista (slicing): {sub_lista}")

# Iterar sobre listas (ya visto en Patrón 1.6)

```

Explicación:

- **Mutable:** Se pueden cambiar sus elementos después de creadas.

- **Ordenadas:** Los elementos mantienen el orden de inserción.
- **Índices:** Acceso a elementos por su posición (0-basado).
- **Métodos:** `append()`, `insert()`, `remove()`, `pop()`, `sort()` son esenciales.

Patrón 4.2: Tuplas (Colecciones Ordenadas e Inmutables)

Las tuplas son similares a las listas, pero una vez creadas, sus elementos no pueden ser modificados.

```
# Creación de tuplas
tupla_vacia = ()
coordenadas = (10.0, 20.5) # Tupla de dos elementos
colores_rgb = ("rojo", "verde", "azul")

# Tupla de un solo elemento (¡importante la coma!)
tupla_un_elemento = ("solo",)

print(f"\n--- Tuplas ---")
print(f"Coordenadas: {coordenadas}")

# Acceso a elementos (por índice)
x = coordenadas[0]
print(f"Coordenada X: {x}")

# Intentar modificar una tupla generará un error (TypeError)
# coordenadas[0] = 15.0 # Esto daría un error

# Desempaquetado de tuplas
nombre, anio = ("Python", 1991)
print(f"Lenguaje: {nombre}, Año: {anio}")
```

Explicación:

- **Inmutables:** No se pueden añadir, eliminar o modificar elementos una vez creada.
- **Ordenadas:** Mantienen el orden de inserción.
- **Uso:** Ideales para datos que no deben cambiar (coordenadas, registros de base de datos, claves de diccionario compuestas).

Patrón 4.3: Diccionarios (Colecciones No Ordenadas de Pares Clave-Valor)

Los diccionarios almacenan datos en pares `clave: valor` , donde cada clave es única.

```
# Creación de diccionarios
diccionario_vacio = {}
persona = {
    "nombre": "Ana",
    "edad": 30,
    "ciudad": "Madrid"
}

print(f"\n--- Diccionarios ---")
print(f"Datos de persona: {persona}")

# Acceso a valores (por clave)
nombre_persona = persona["nombre"]
print(f"Nombre: {nombre_persona}")

# Modificación y adición de elementos
persona["edad"] = 31 # Modificar valor existente
persona["profesion"] = "Ingeniera" # Añadir nuevo par clave-valor
print(f"Persona actualizada: {persona}")

# Métodos comunes de diccionarios
claves = persona.keys() # Obtener todas las claves
valores = persona.values() # Obtener todos los valores
items = persona.items() # Obtener todos los pares clave-valor (como tuplas)

print(f"Claves: {list(claves)}")
print(f"Valores: {list(valores)}")
print(f"Items: {list(items)}")

# Eliminar elementos
del persona["ciudad"] # Eliminar por clave
print(f"Después de eliminar ciudad: {persona}")

# Usar .get() para evitar errores si la clave no existe
ocupacion = persona.get("ocupacion", "Desconocida") # Devuelve "Desconocida"
si no existe
print(f"Ocupación: {ocupacion}")

# Iterar sobre diccionarios
print("\nIterando sobre claves:")
for clave in persona.keys(): # O simplemente for clave in persona:
    print(clave)

print("\nIterando sobre valores:")
for valor in persona.values():
```



```
print(valor)

print("\nIterando sobre items (clave, valor):")
for clave, valor in persona.items():
    print(f"{clave}: {valor}")
```

Explicación:

- **Clave-Valor:** Cada elemento es un par donde la `clave` es única e inmutable (generalmente cadenas o tuplas) y el `valor` puede ser cualquier tipo de dato.
- **Mutables:** Se pueden añadir, eliminar o modificar pares clave-valor.
- **Acceso:** Se accede a los valores usando las claves entre corchetes (`[]`) o con el método `get()` .
- **Métodos:** `keys()` , `values()` , `items()` son muy útiles para iterar o obtener listados de sus componentes.

Patrón 4.4: Conjuntos (Colecciones No Ordenadas de Elementos Únicos)

Los conjuntos son colecciones de elementos únicos y no ordenados. Son útiles para eliminar duplicados y realizar operaciones matemáticas de conjuntos.

```

# Creación de conjuntos
conjunto_vacio = set() # Importante: {} crea un diccionario vacío
numeros_primos = {2, 3, 5, 7, 11}
letras = {"a", "b", "c", "a"} # Los duplicados se eliminan automáticamente

print(f"\n--- Conjuntos ---")
print(f"Conjunto de letras: {letras}") # Salida: {"a", "b", "c"}

# Añadir y eliminar elementos
letras.add("d")
print(f"Después de add: {letras}")

letras.remove("b") # Genera error si el elemento no existe
print(f"Después de remove: {letras}")

letras.discard("z") # No genera error si el elemento no existe
print(f"Después de discard: {letras}")

# Operaciones de conjuntos
conjunto_a = {1, 2, 3, 4}
conjunto_b = {3, 4, 5, 6}

union = conjunto_a.union(conjunto_b) # {1, 2, 3, 4, 5, 6}
interseccion = conjunto_a.intersection(conjunto_b) # {3, 4}
diferencia = conjunto_a.difference(conjunto_b) # {1, 2}
diferencia_simetrica = conjunto_a.symmetric_difference(conjunto_b) # {1, 2, 5, 6}

print(f"Conjunto A: {conjunto_a}, Conjunto B: {conjunto_b}")
print(f"Unión: {union}")
print(f"Intersección: {interseccion}")
print(f"Diferencia (A-B): {diferencia}")
print(f"Diferencia simétrica: {diferencia_simetrica}")

# Comprobar pertenencia
es_miembro = 3 in conjunto_a # True
print(f"¿3 está en conjunto_a? {es_miembro}")

```

Explicación:

- **Elementos únicos:** No permiten duplicados.
- **No ordenados:** Los elementos no tienen un orden específico.
- **Mutables:** Se pueden añadir o eliminar elementos.

- **Operaciones:** Ideales para operaciones como unión, intersección, diferencia, etc.

Continuaremos con los patrones para UT5/UT6 en la siguiente sección, que abordarán el manejo de excepciones, funciones avanzadas y el uso de librerías. Esto te proporcionará una base sólida para el RA3, especialmente en la parte de depuración y estructuras de control y datos.

UT5: Excepciones y Manejo de Errores (RA3 - Parte III)

El manejo de excepciones es crucial para crear programas robustos que no fallen ante entradas inesperadas o situaciones anusuales. Permite controlar el flujo del programa cuando ocurren errores.

Patrón 5.1: Manejo Básico de Excepciones (`try-except`)

El bloque `try-except` permite “intentar” ejecutar un código y “capturar” los errores que puedan ocurrir.

```
def dividir_numeros(a, b):
    try:
        resultado = a / b
        print(f"El resultado de la división es: {resultado:.2f}")
    except ZeroDivisionError:
        print("Error: No se puede dividir por cero.")
    except TypeError:
        print("Error: Asegúrate de que ambos valores sean números.")
    except Exception as e: # Captura cualquier otra excepción
        print(f"Ha ocurrido un error inesperado: {e}")

if __name__ == "__main__":
    dividir_numeros(10, 2) # División normal
    dividir_numeros(10, 0) # Error de división por cero
    dividir_numeros(10, "dos") # Error de tipo
    dividir_numeros("diez", 2) # Otro error de tipo
```

Explicación:

- `try`: Contiene el código que podría generar una excepción.

- `except TipoDeError` : Captura una excepción específica. Puedes tener múltiples bloques `except` .
- `except Exception as e` : Captura cualquier tipo de excepción no especificada y guarda el mensaje de error en la variable `e` .

Patrón 5.2: Bloque `finally`

El bloque `finally` se ejecuta siempre, ocurra o no una excepción. Es útil para liberar recursos (cerrar archivos, conexiones de red, etc.).

```
def leer_archivo(nombre_archivo):
    f = None # Inicializar f a None por si hay error antes de abrir
    try:
        f = open(nombre_archivo, "r")
        contenido = f.read()
        print(f"Contenido del archivo:\n{contenido}")
    except FileNotFoundError:
        print(f"Error: El archivo '{nombre_archivo}' no se encontró.")
    except Exception as e:
        print(f"Ha ocurrido un error al leer el archivo: {e}")
    finally:
        if f: # Asegurarse de que el archivo se abrió antes de intentar
            cerrarlo
            f.close()
        print("Archivo cerrado (desde finally).")

if __name__ == "__main__":
    # Crear un archivo de prueba
    with open("mi_archivo.txt", "w") as f:
        f.write("Hola, esto es una prueba.\n")
        f.write("Línea dos.")

    leer_archivo("mi_archivo.txt")
    print("\n")
    leer_archivo("archivo_inexistente.txt")
```

Explicación:

- `finally` : El código dentro de este bloque se ejecuta siempre, sin importar si el `try` se completó con éxito o si una excepción fue capturada.
- Es ideal para tareas de limpieza.

Patrón 5.3: Lanzar Excepciones (raise)

`raise` permite forzar la ocurrencia de una excepción, lo cual es útil para indicar condiciones de error en tu propio código.

```
def procesar_edad(edad):
    if not isinstance(edad, (int, float)):
        raise TypeError("La edad debe ser un número.")
    if edad < 0 or edad > 120:
        raise ValueError("La edad debe estar entre 0 y 120 años.")
    print(f"Edad procesada correctamente: {edad} años.")

if __name__ == "__main__":
    try:
        procesar_edad(30)
        procesar_edad(-5) # Esto lanzará un ValueError
        procesar_edad("veinte") # Esto lanzará un TypeError
    except ValueError as ve:
        print(f"Error de valor: {ve}")
    except TypeError as te:
        print(f"Error de tipo: {te}")
```

Explicación:

- `raise Excepcion("Mensaje")` : Crea y lanza una excepción con un mensaje personalizado.
- Útil para validar entradas o estados internos del programa.

UT6: Funciones y Librerías (RA3 - Parte IV)

Esta unidad profundiza en las funciones, incluyendo argumentos más avanzados, funciones anónimas (lambda) y la importancia de las librerías para extender la funcionalidad de Python.

Patrón 6.1: Funciones con Argumentos Variables (*args y **kwargs)

Permiten que una función acepte un número variable de argumentos posicionales (`*args`) o de argumentos de palabra clave (`**kwargs`).

```
def sumar_todos(*numeros):
    """Suma un número variable de argumentos posicionales."""
    total = 0
    for num in numeros:
        total += num
    return total

def configurar_perfil(nombre, **opciones):
    """Configura un perfil con opciones variables de palabra clave."""
    print(f"Configurando perfil para: {nombre}")
    for clave, valor in opciones.items():
        print(f"  - {clave}: {valor}")

if __name__ == "__main__":
    print("\n--- Funciones con *args y **kwargs ---")
    print(f"Suma de 1, 2, 3: {sumar_todos(1, 2, 3)}")
    print(f"Suma de 5, 10, 15, 20: {sumar_todos(5, 10, 15, 20)}")

    configurar_perfil("Juan", edad=25, ciudad="Barcelona",
email="juan@example.com")
    print("\n")
    configurar_perfil("María", profesion="Diseñadora")
```

Explicación:

- `*args` : Recopila un número arbitrario de argumentos posicionales en una tupla.
- `**kwargs` : Recopila un número arbitrario de argumentos de palabra clave en un diccionario.
- Muy útiles para funciones flexibles que pueden recibir diferentes conjuntos de datos.

Patrón 6.2: Funciones Lambda (Funciones Anónimas)

Las funciones lambda son pequeñas funciones anónimas (sin nombre) que pueden tener cualquier número de argumentos, pero solo una expresión. Son útiles para operaciones sencillas y de una sola línea.

```

# Lambda para sumar dos números
sumar_lambda = lambda a, b: a + b
print(f"\n--- Funciones Lambda ---")
print(f"Suma con lambda (5, 3): {sumar_lambda(5, 3)}")

# Lambda para verificar si un número es par
es_par_lambda = lambda num: num % 2 == 0
print(f"¿Es 4 par? {es_par_lambda(4)}")
print(f"¿Es 7 par? {es_par_lambda(7)}")

# Uso de lambda con funciones de orden superior (map, filter, sorted)
lista_numeros = [1, 2, 3, 4, 5, 6]

# Filtrar números pares
pares = list(filter(lambda x: x % 2 == 0, lista_numeros))
print(f"Números pares: {pares}")

# Elevar al cuadrado cada número
cuadrados = list(map(lambda x: x**2, lista_numeros))
print(f"Números al cuadrado: {cuadrados}")

# Ordenar una lista de diccionarios por una clave
personas = [
    {"nombre": "Ana", "edad": 30},
    {"nombre": "Juan", "edad": 25},
    {"nombre": "María", "edad": 35}
]
personas_ordenadas = sorted(personas, key=lambda p: p["edad"])
print(f"Personas ordenadas por edad: {personas_ordenadas}")

```

Explicación:

- `lambda` argumentos: `expresion`: Sintaxis concisa para funciones simples.
- Comúnmente usadas como argumentos para funciones de orden superior (`map()`, `filter()`, `sorted()`) que esperan una función.

Patrón 6.3: Uso de Módulos y Librerías Estándar

Python tiene una vasta biblioteca estándar. Saber cómo importar y usar módulos es esencial.

```

import math # Módulo para operaciones matemáticas
import random # Módulo para generar números aleatorios (ya visto)
import datetime # Módulo para trabajar con fechas y horas

print("\n--- Uso de Módulos y Librerías ---")
# Uso del módulo math
print(f"Valor de PI: {math.pi}")
print(f"Raíz cuadrada de 16: {math.sqrt(16)}")

# Uso del módulo datetime
fecha_actual = datetime.date.today()
hora_actual = datetime.datetime.now().time()
print(f"Fecha actual: {fecha_actual}")
print(f"Hora actual: {hora_actual}")

# Formatear fecha
fecha_formateada = fecha_actual.strftime("%d/%m/%Y")
print(f"Fecha formateada: {fecha_formateada}")

# Crear un objeto datetime específico
fecha_nacimiento = datetime.datetime(1990, 5, 15, 10, 30, 0)
print(f"Fecha de nacimiento: {fecha_nacimiento}")

# Calcular diferencia entre fechas
diferencia = datetime.datetime.now() - fecha_nacimiento
print(f"Han pasado {diferencia.days} días desde el nacimiento.")

# Importar solo una parte de un módulo
from collections import Counter

lista_elementos = ["a", "b", "a", "c", "b", "a"]
conteo = Counter(lista_elementos)
print(f"Conteo de elementos: {conteo}")

```

Explicación:

- `import modulo`: Importa un módulo completo.
- `from modulo import funcion_o_clase`: Importa solo partes específicas de un módulo.
- La biblioteca estándar de Python es muy extensa; familiarizarse con módulos como `math`, `random`, `datetime`, `os`, `sys`, `json`, `collections` es muy útil.

Con estos patrones, tienes una base sólida para abordar los exámenes de las unidades UT1 a UT6. Recuerda practicar cada patrón y entender cuándo aplicarlo. ¡Mucha suerte!

Conclusión

Esta guía ha cubierto los patrones de código esenciales para las unidades didácticas UT1 a UT6, abarcando los Resultados de Aprendizaje RA1, RA2 y RA3. Al dominar estos patrones, no solo comprenderás los fundamentos de la programación en Python, sino que también desarrollarás la capacidad de escribir código estructurado, eficiente y robusto, habilidades cruciales para cualquier desarrollador.

Recuerda que la práctica constante es clave. Intenta modificar estos ejemplos, combinarlos y aplicarlos a nuevos problemas. La depuración de código, que es una parte fundamental del RA3, se mejora significativamente al entender cómo funcionan estas estructuras y cómo se comportan ante diferentes entradas. Utiliza un entorno de desarrollo integrado (IDE) y sus herramientas de depuración para seguir el flujo de ejecución y entender el estado de las variables en cada paso.

¡Te deseo mucho éxito en tus exámenes y en tu camino como programador!

Referencias

- [Documentación oficial de Python](#)
- [PEP 8 – Style Guide for Python Code](#)
- [Real Decreto ⁴⁰⁵/2023, de 29 de mayo \(Currículo DAM/DAW\)](#)